Edge Detection for Autonomous Systems on Azure Cloud

QUANTUM IMAGE PROCESSING

Quantum Software II · 2025
Master of Quantum Computing, University of Calgary

Presented by

Tomomi Nakamura

About Me

4 years in industry software + quantum computing graduate — bridging production engineering and quantum research

Current	Software Developer @1QBit, Canada
Industry Background	Cloud Integration Engineer @Avanade, Tokyo
Education	Master of Quantum Computing @University of Calgary BS in Computer Science & Software Engineering @University of Washington
Software Skills	Python, C#, C++, SQL, Azure, CI/CD
Quantum Skills	Quantum Algorithm, Compilation, Error Correction
Certifications	D-Wave Ocean, Azure Developer Associate, Azure Fundamentals

Content

Project Background

- Quantum-enhanced edge detection for autonomous vehicles and aviation.
- Inspired by Airbus × BMW Group Quantum Computing Challenge 2024.

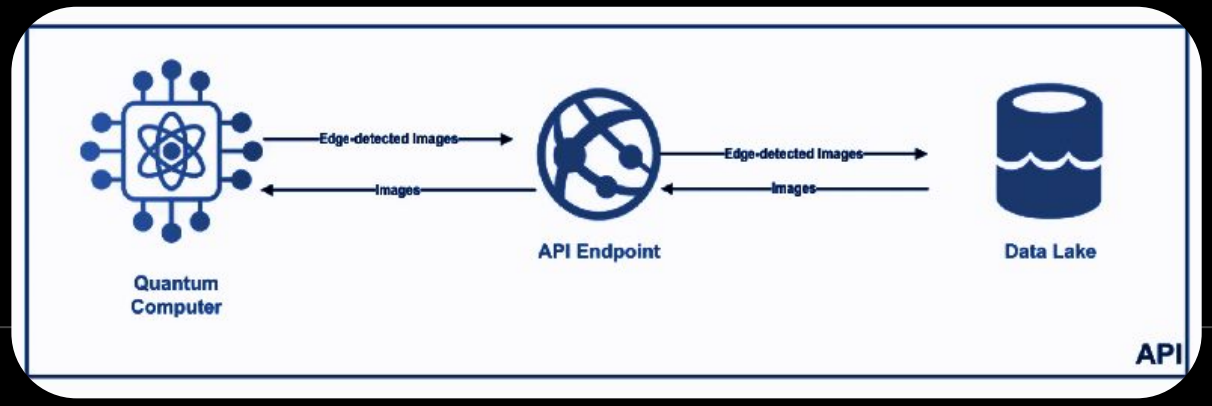
Classical method	Reliable but hitting performance limits at scale
Data demands grow	More sensors, higher resolution, faster decisions matter
Quantum advantage	Potential to process beyond classical boundaries

Scope

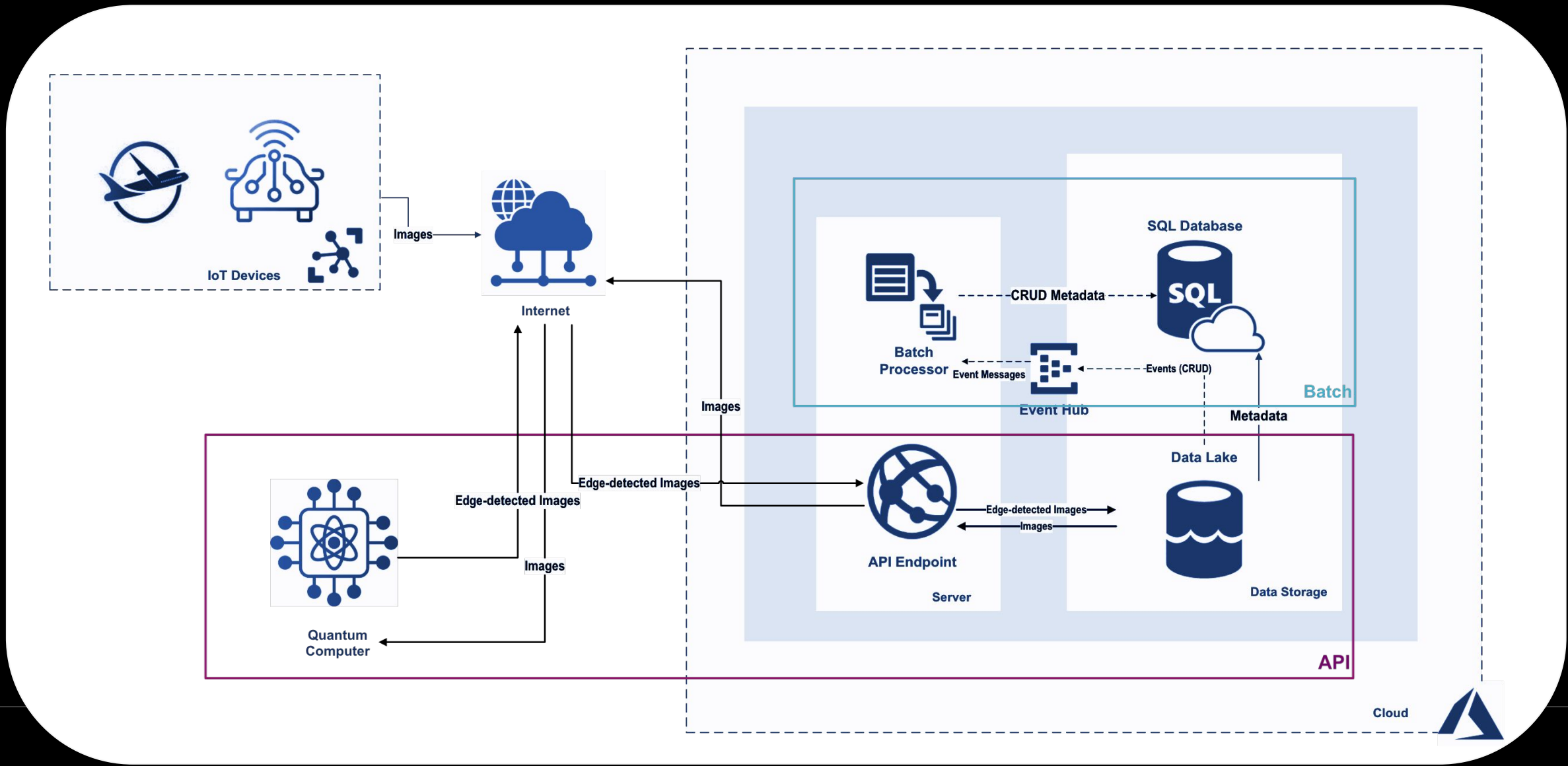
	What We Built	Contribution
Quantum Algorithm	QPIE encoding + QHED edge detection circuits	Partner
Classical Comparison	OpenCV Canny edge detection for comparison	Partner
Cloud Infrastructure	Azure IoT pipeline design, API implementation	Me
Cost Estimation	Storage cost by daily image volume (Power BI)	Me
End-to-End Integration	Azure (input) → IBM Quantum (processing) → Azure (output)	Collaborated
Real Hardware	IBM Quantum execution	Collaborated

Constraints & Design Decisions

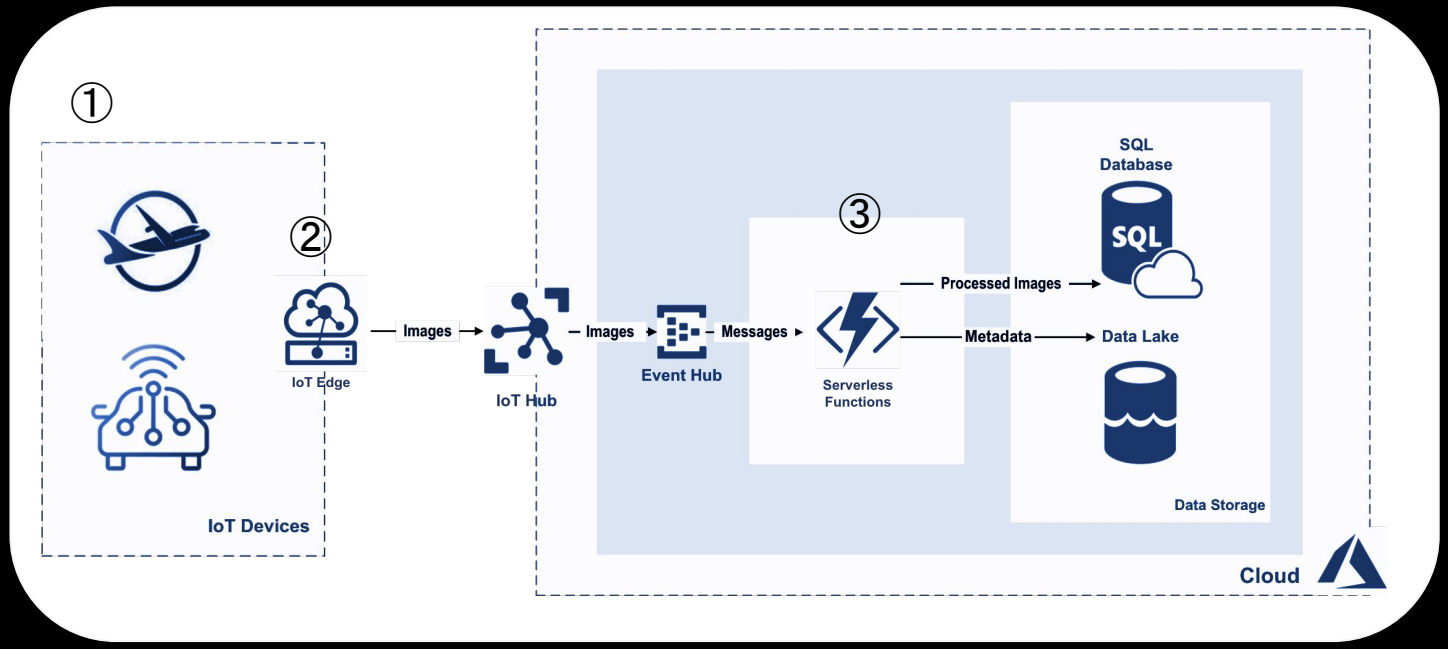
Constraint	Impact	What We Did Instead
10 min IBM Quantum access	No batch processing	Selected subset of runway images
Azure Student Tier	No IoT pipeline,	Stored dataset manually in Data Lake
Dataset only — no live sensors	No real-time event-driven triggers	Designed architecture for future extension
NISQ/QPU cost models not yet standardized	No meaningful QPU vs classical cost comparison	Focused on storage by daily image volume



Overall Infrastructure Design



Data Cleansing Design



{image01, timestamp, airport, weather, time_of_day, image_path}

{image02, timestamp, airport, weather, time_of_day, image_path}

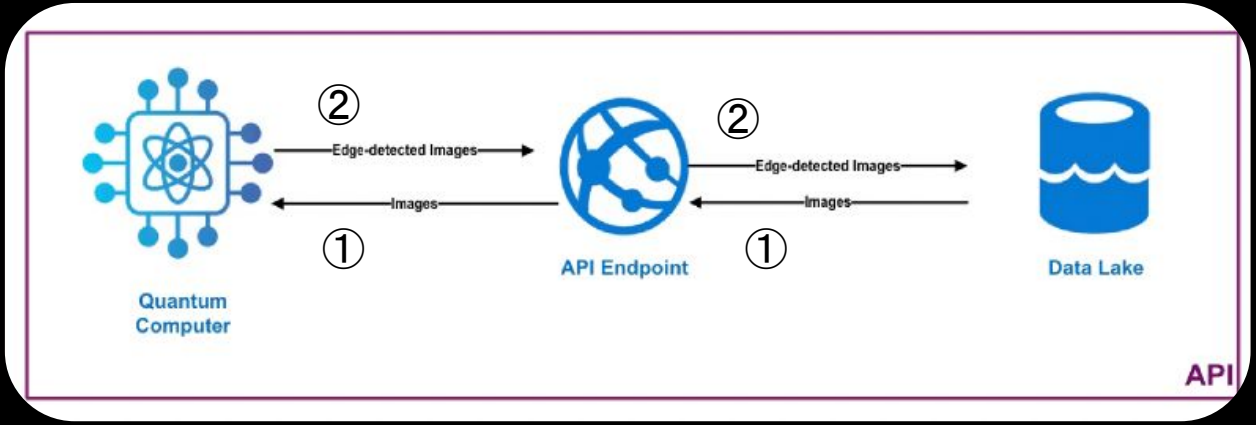
① Raw Images

② Filtered + Compressed

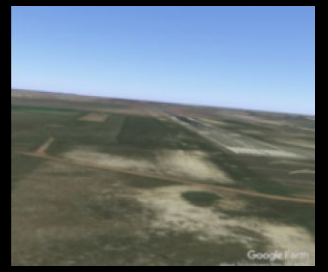
③ Metadata added

API Implementation

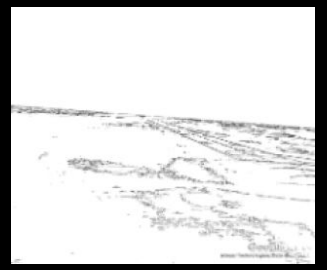
API	Purpose	URL	Header
GET	List of images in dataset.	https://qip-dev-fye9bwewd2daath7.canadacentral-01.azurewebsites.net/images	-
GET	Retrieve an image.	https://qip-dev-fye9bwewd2daath7.canadacentral-01.azurewebsites.net/images/{image_name}	- ← ①
POST	Upload the edge-detected image back.	https://qip-dev-fye9bwewd2daath7.canadacentral-01.azurewebsites.net/images/upload	Content-Type: multipart/form-data ← ②



① Original

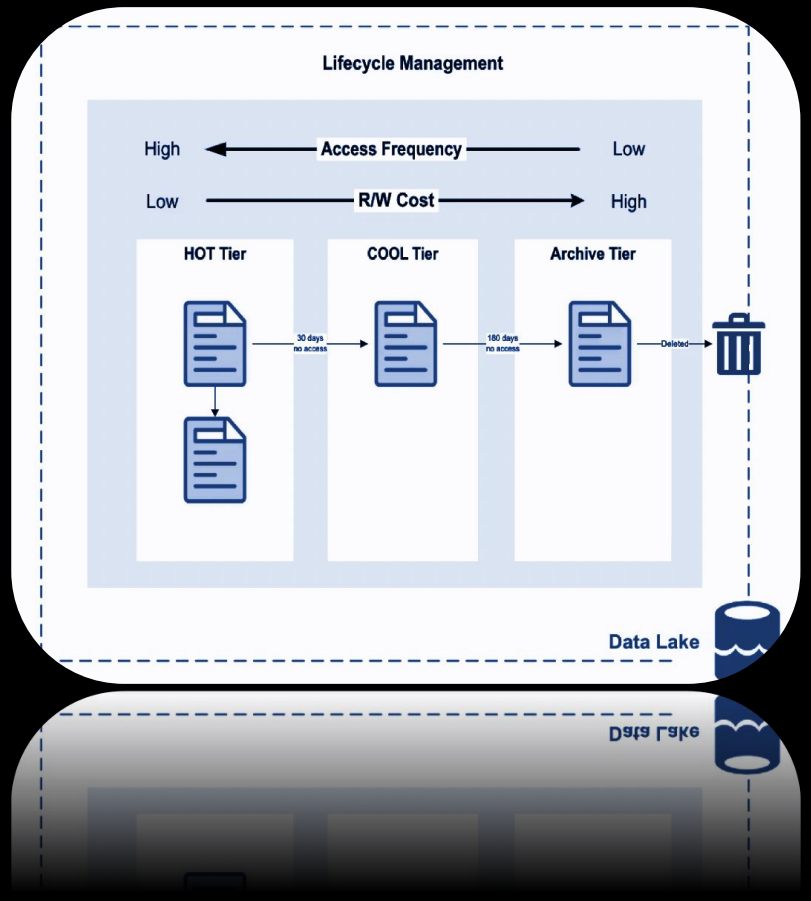


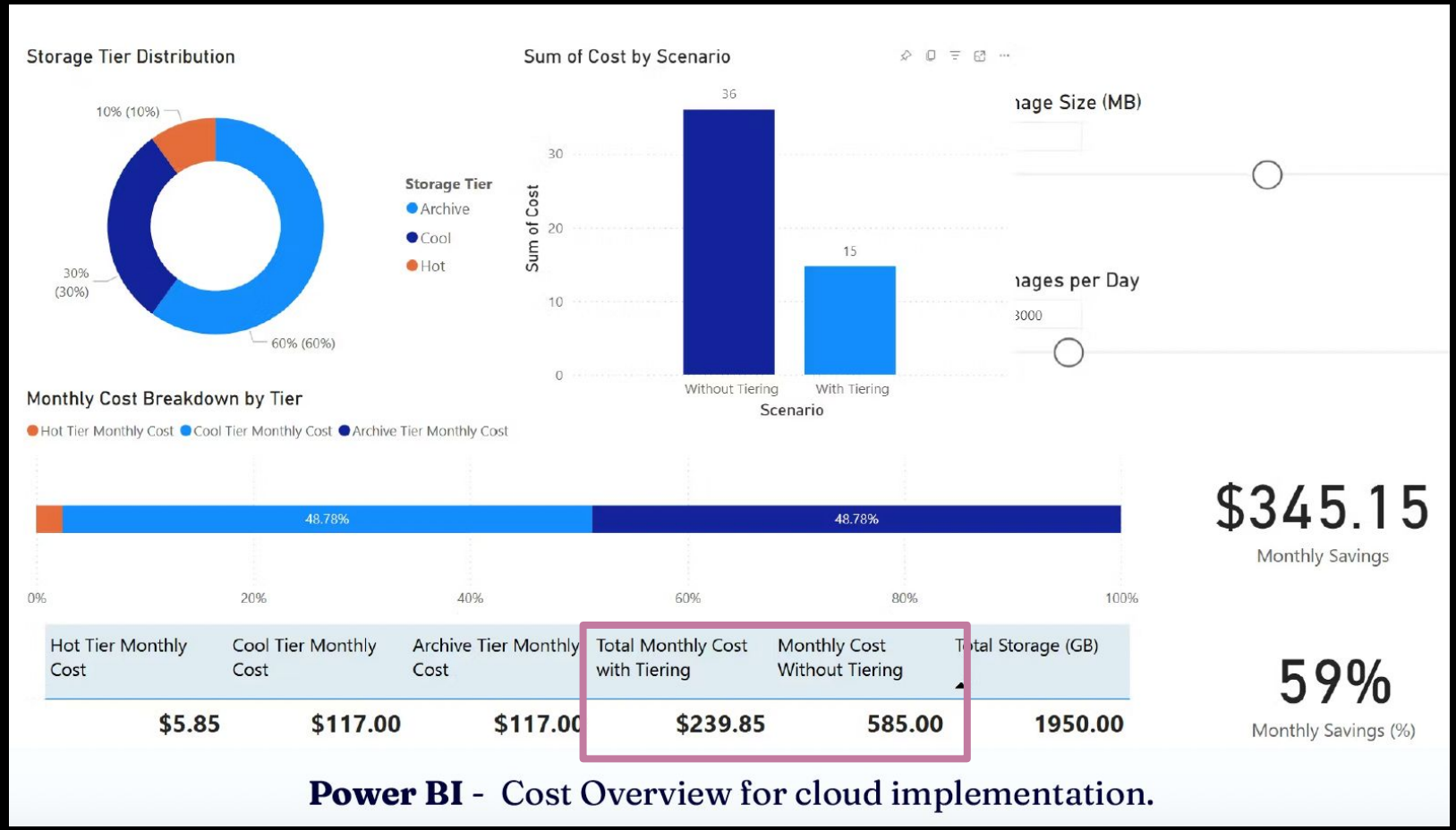
② Edge-detected



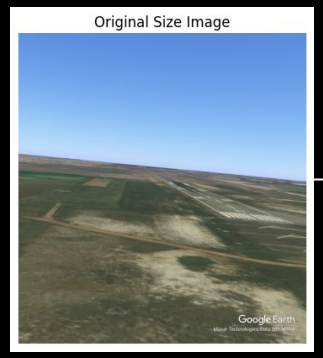
https://github.com/tomomii0806/quantum-image-processing-api

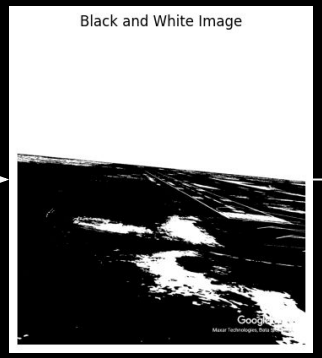
Storage Cost Estimation

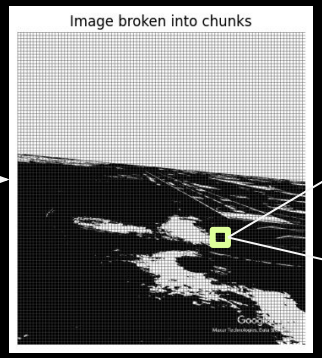


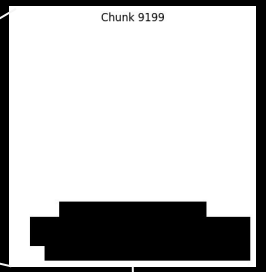


Quantum Image Processing





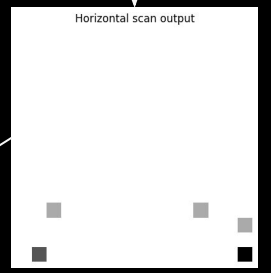




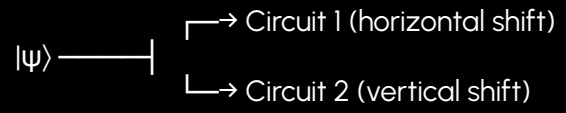
Quantum Probability Image Encoding (QPIE)

- *pixels into quantum state*

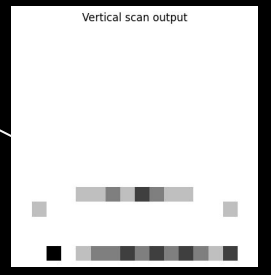
[0, 128, 255, 64, ...] → $|\psi\rangle = 0|000\rangle + 0.426|001\rangle + 0.849|010\rangle + \ldots$



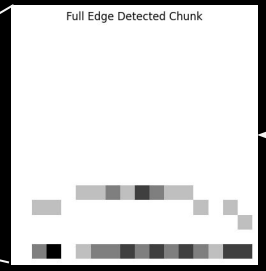
Quantum Hadamard Edge Detection (QHED)

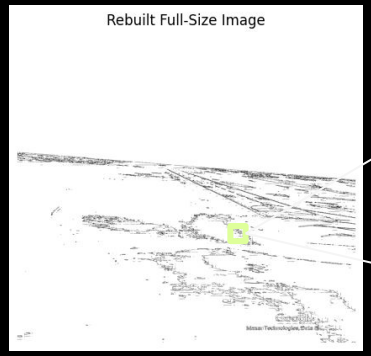


- high probability = edge
- low probability = no edge



Combined





https://github.com/FGranda2/Quantum-Image-Processing

Quantum Advantage & Result

Result: 9 Qubit Circuit

	Local Simulator		IBM QC (ibm_sherbrooke)	Classical (OpenCV)
Chunk	9200 	1 	1 	full image
Performance	~6 min	~41 ms	~195 sec	~10 Seconds
Shots (per chunk)	1000	1000	1000	-

Advantage : Space Complexity

Pixels (N)	Classical O(N) (greyscale)	Quantum O(log N)
32 pixels	≈ 32 bytes	5 qubits
1,024 pixels	≈ 1,024 bytes	10 qubits
1M pixels	≈ 1M bytes	20 qubits
Full HD	≈ 2M bytes	21 qubits

- End-to-end pipeline validated on real IBM quantum hardware
- Full image processed successfully on local simulator
- Classical method still faster today
  - NISQ era hardware limitation
- O(log N) space advantage
  - promising for high-resolution aviation imaging

https://github.com/tomomii0806/quantum-image-processing-api

Future Improvement & On-going Study

Quantum Compilation — Toward Fault-tolerant Quantum

Current State

NISQ only

- Used Qiskit default transpiration
- IBM native gate sets:
  - {CX, I, RZ, SX, X, etc…}
- Simple gate commutativity optimization

RZ(θ) → RZ(θ) = VERY NOISY !

Future Direction

FTQC

- Custom Clifford+T compiler
- Universal gate sets:
  - {H, S, CNOT, T, $T^{\dagger}$, etc…}
- Minimize T-gate count
  - Each T-gate = magic state distillation ~15:1 overhead

RZ(θ) → T · H · $T^{\dagger}$ · H · T · H · $T^{\dagger}$ = LESS NOISE !

↓

minimize T-count (Gridsynth algorithm)

My Ongoing Study

Quantum compiler demo in Rust

Current stage:

- Commutativity-based gate optimization

Goal:

- Understand compiler internals at a systems level
- Extend toward T-gate aware synthesis

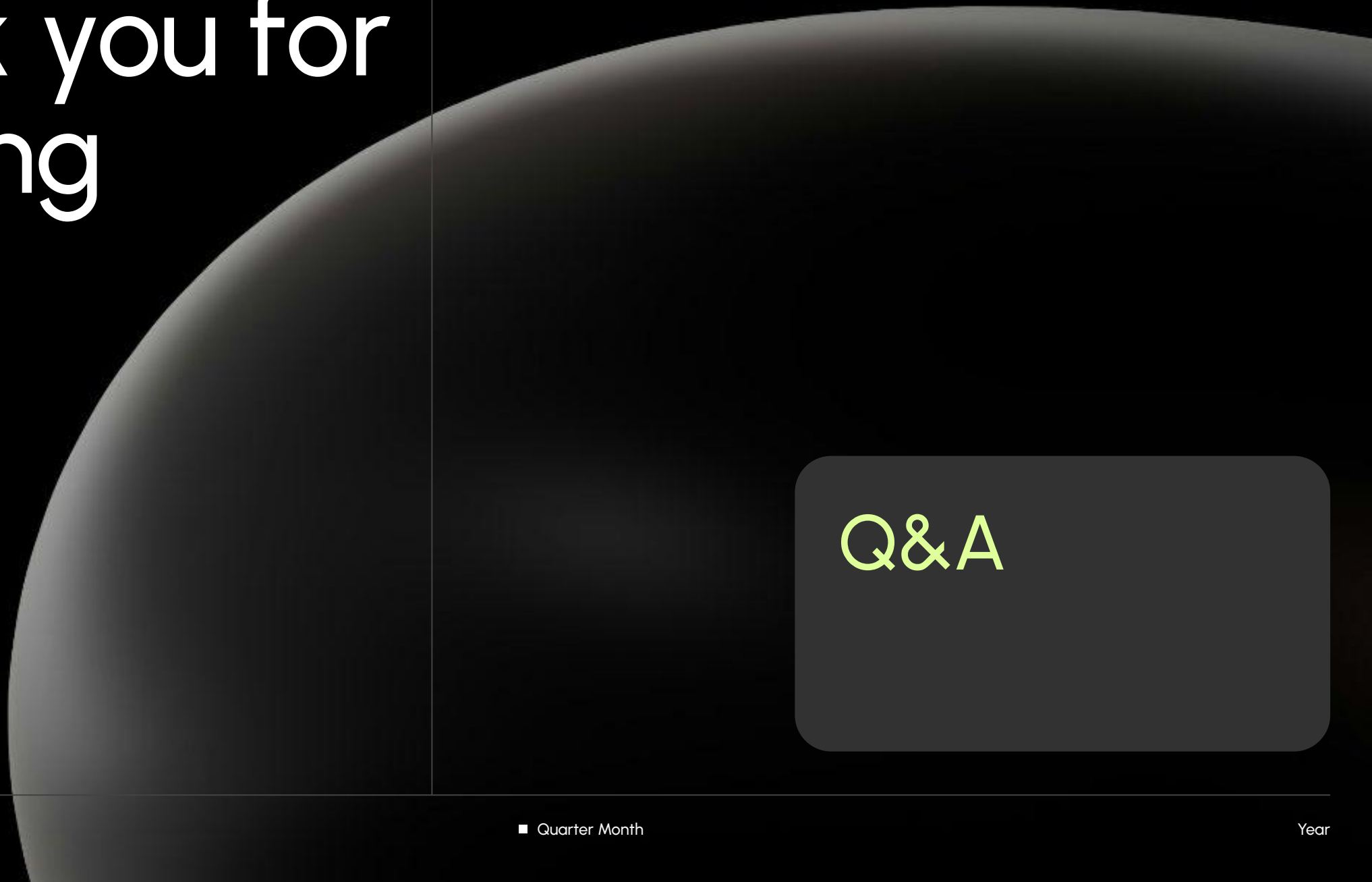
Why Rust?

- Performance critical
- Memory safety
- Industry interest in systems-level quantum tooling

Skill Demonstrated

Research Mindset	Implement academic algorithms into production code
Quantum Knowledge	Encoding, circuit design, noise, NISQ → FTQC
Systems-Level Thinking	Compiler internals, gate optimization, Rust
Software Engineering	End-to-end pipeline — production minded
Cloud & Infrastructure	Azure, REST API, cost modelling
Benchmarking & Analysis	Classical vs quantum evaluation
Research ↔ Engineering Bridge	Mix of quantum & software background

Thank you for listening



Q&A